

Boids Parallelization

Matteo Tinacci

matteo.tinacci@edu.unifi.it

Abstract

This project analyzes the performance optimization of a Boids simulation, whose brute-force flocking algorithm presents a computational bottleneck with a complexity of $O(N^2)$. The adopted approach was shared-memory parallelization using OpenMP, comparing different parallelization strategies (targeting Stage 1, Stage 2, or both) and various scheduling policies (static vs. dynamic). The key results, obtained on an 8-core CPU, show a maximum speedup of $4.63\times$ with 8 threads. The analysis demonstrates that the speedup is limited by Amdahl's Law and that Hyper-Threading (16 threads) is counterproductive. Furthermore, the data reveal that CPU cache locality had a greater impact on performance than thread management overhead.

Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

The *Boids* simulation [1] is used to realistically model the collective motion of bird flocks, fish schools, or crowds. The overall complex behavior emerges from the application of three simple rules that each agent (or “boid”) follows in a decentralized manner:

1. **Separation:** Avoid crowding with nearby neighbors.
2. **Alignment:** Match the direction of motion to the average heading of nearby agents.
3. **Cohesion:** Move toward the average position of nearby agents.

1.1. The Computational Problem

The most straightforward implementation—the *brute-force* approach—presents a significant performance issue. To correctly apply the three rules, each boid must compute its distance and relative velocity with respect to every other boid in the simulation.

This approach introduces a computational bottleneck with a complexity of $O(N^2)$, where N is the number of boids.

Such quadratic growth makes the brute-force simulation unsustainable for a large number of agents, limiting its usefulness in complex scenarios. However, this *embarrassingly parallel* nature—where the computation of each boid is (initially) independent of the others—makes it an ideal candidate for optimization through parallel computing.

1.2. Objective

The goal of this project is to optimize the $O(N^2)$ simulation by exploiting shared-memory parallelism provided by OpenMP. Various strategies for parallelizing the workload across a multi-core CPU are analyzed, measuring their impact on performance and scalability. The analysis focuses not only on reducing execution time (speedup), but also on the limits imposed by Amdahl's Law, thread contention (Hyper-Threading), and advanced hardware factors such as cache locality.

2. Methodologies and Test Environment

To ensure the reproducibility of the results, all the resources and settings used for the experiments are reported here.

2.1. Hardware & Software

All benchmarks were executed on the same machine to guarantee data consistency. The specifications of the test environment are as follows:

1. **Hardware:** HP Pavilion Power Laptop 15-cb0xx
2. **CPU:** 8 Physical Cores
3. **Operating System:** Ubuntu 22.04
4. **Compiler:** g++ (GCC) with OpenMP support
5. **Languages:** C++ (simulation), Python (data analysis)
6. **Libraries:** OpenMP, SFML (visualization only), Pandas, Matplotlib

2.2. Benchmark Methodologies

The primary objective of the test is to measure *strong scaling*: how performance changes as the number of processors (threads) increases, while keeping the problem size fixed.

1. **Fixed Problem Size:** All tests were executed with the same workload:
 - (a) **Number of Boids:** `NUM_BOIDS = 5000`
 - (b) **Number of Iterations:** `NUM_STEPS = 1000`
2. **Time Measurement:** The OpenMP high-resolution timer `omp_get_wtime()` was used. Timers were strategically placed to measure only the computation loop, excluding all I/O costs (console output) and initialization overhead (flock creation and memory allocation).
3. **Test Automation:** Scalability tests were automated using a Bash script (`run_benchmarks.sh`). The script executed the `BenchmarkMain` binary while setting the environment variable `OMP_NUM_THREADS` to 1, 2, 4, 8, and 16.

4. **Data Analysis:** Results were saved to a `.csv` file and subsequently analyzed with a Python script using the Pandas library (for calculating speedup and efficiency) and Matplotlib (for generating plots).

Given the prohibitive runtime of a single complete test cycle (approximately 3.5 hours on the test machine), only one measurement was performed for each configuration. This decision is justified by the nature of the observed results:

1. **Strong Signal:** The measured “signal” (i.e., the performance difference between the sequential and parallel versions) is not on the order of milliseconds, but hundreds of seconds (e.g., 760 s vs. 170 s).
2. **Negligible Noise:** This signal is several orders of magnitude greater than the expected statistical noise introduced by the operating system (1–3 s).

It is therefore assumed that the results from a single controlled execution (with all background processes closed) are sufficiently robust and reliable to demonstrate scalability concepts, Amdahl’s Law, and the impact of thread contention.

3. Algorithm Description

The implementation of the Boids simulation in this project is based on a discrete update loop executed for a fixed number of `NUM_STEPS`. The core characteristic of the algorithm is a clear separation between the computation of behavioral rules and the application of movement. The algorithm was built using the Hunter Adams’[2] one as reference.

3.1. Algorithm Structure

Within each simulation step, the computation is strictly divided into two sequential phases (or “stages”):

1. **STAGE 1 – Rule Computation** (*calculateRules*): This is the computationally intensive phase. For each individual boid in the flock, the program iterates over all

other boids to apply the three rules (Separation, Alignment, Cohesion). The result of this process is an acceleration vector (`boid.acceleration`) that determines the desired movement direction for that boid.

2. **STAGE 2 – State Update** (*updateState*): This is a lightweight phase. After all boids have computed their desired acceleration, a second loop iterates through the entire flock. In this phase, each boid applies its computed acceleration to update its velocity and, subsequently, its position (`boid.velocity` and `boid.position`).

3.2. Bottleneck Analysis

The two-phase division is essential for maintaining simulation correctness and preventing **data hazards**. If one boid (Boid A) were to compute its acceleration and immediately update its position (S1+S2), another boid (Boid B), still in the middle of its computation (S1), would “read” the new position of Boid A. This *dirty read* would break the integrity of the simulation step, as computations would no longer be performed on a consistent global state.

Therefore, two separate loops are employed:

1. The first loop (STAGE 1) represents the computational bottleneck. With N boids, each boid performs N checks, leading to an overall complexity of $O(N^2)$. This portion of the code accounts for the vast majority of the execution time.
2. The second loop (STAGE 2) is a simple $O(N)$ pass. It is fast, but it cannot begin until STAGE 1 has been completed.

4. Parallelization Strategies

Based on the two-phase structure (Stage 1 and Stage 2) of the algorithm, five different execution strategies were implemented and compared to evaluate the impact of OpenMP parallelization.

4.1. Sequential (Baseline)

The first version is the pure *brute-force* implementation. Both loops (Stage 1 and Stage 2) are executed on a single thread. This code contains no OpenMP directives and serves as the baseline for computing the speedup of all other strategies.

4.2. Parallel (Stage 1 Only)

This strategy focuses on optimizing the bottleneck identified in Stage 1.

1. **Stage 1:** Parallelized using the `#pragma omp parallel for` directive. The rule computation workload is distributed among all available threads.
2. **Stage 2:** Remains sequential.
3. **Objective:** This test directly demonstrates Amdahl’s Law. A significant speedup is expected, but it will be limited by the fraction of time still spent in the sequential Stage 2.

4.3. Parallel (Stage 2 Only)

This is a “control” strategy implemented to validate the bottleneck analysis.

1. **Stage 1:** Remains sequential.
2. **Stage 2:** Parallelized using `#pragma omp parallel for`.
3. **Objective:** The hypothesis is that this strategy will yield no benefit (or even degrade performance). Leaving the $O(N^2)$ bottleneck untouched while adding OpenMP overhead to the already lightweight $O(N)$ portion should prove ineffective.

4.4. Parallel (Stage 1 + Stage 2, Separate Regions)

This strategy parallelizes both computational stages using two separate consecutive `parallel for` directives.

1. **Stage 1:** Parallelized with `#pragma omp parallel for`.
2. **Stage 2:** Parallelized with a second `#pragma omp parallel for`.

3. **Objective:** In theory, this strategy incurs the *fork/join* overhead twice—once for each loop. It was tested to evaluate the impact of cache locality under this configuration.

4.5. Parallel (Stage 1 + Stage 2, Single Region)

This is the “textbook” optimization of Strategy 4, designed to reduce overhead.

1. **Stage 1 & Stage 2:** A single `#pragma omp parallel` directive creates the thread team once. Within this region, two `#pragma omp for` directives distribute the work for Stage 1 and Stage 2 among already active threads.
2. **Objective:** The theoretical expectation is that this version will be the fastest, as it parallelizes both stages while paying the overhead only once. The experimental comparison with Strategy 4 reveals the actual impact of this overhead relative to other factors (such as cache locality).

5. Discussion of Results

The first analysis focuses on how execution times vary with the number of threads. The results are shown in Table 3 and in Figure 1.

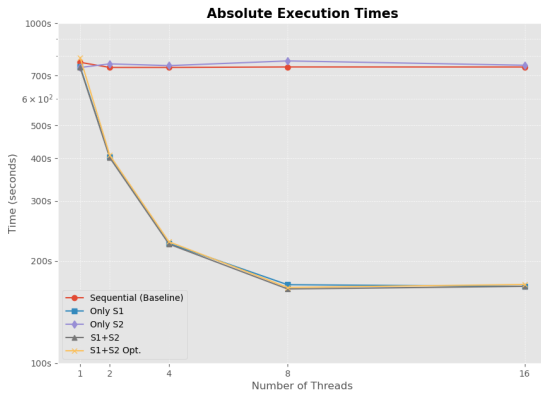


Figure 1. Execution time as a function of the number of threads.

Observing the Sequential, S1, and S2 strategies reveals the following:

1. The **Parallel (Stage 2 Only)** strategy produced even worse execution times than the

sequential baseline in all cases. This clearly demonstrates that Stage 1 accounts for almost the entire computational workload. Parallelizing only Stage 2 is not only ineffective but detrimental, as the OpenMP overhead exceeds the execution time of Stage 2 itself.

2. The **Parallel (Stage 1 Only)** strategy drastically reduced the total runtime to approximately 170 s (with 8 threads). This confirms that the bottleneck was correctly identified and targeted. However, this improvement also exposes the limits imposed by Amdahl’s Law: after optimizing Stage 1, the (relatively small) Stage 2 becomes the new bottleneck. The measured 170 s thus represents $T_{S1,parallel} + T_{S2,seq}$, indicating that further speedup requires Stage 2 to be parallelized as well.

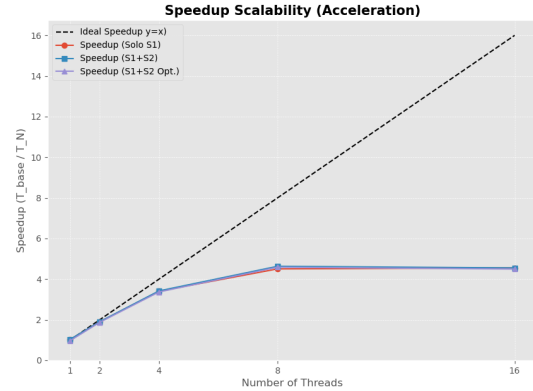


Figure 2. Speedup scalability as the number of threads increases.

By analyzing the speedup and efficiency results shown in Figures 2 and 3, and in Tables 1 and 2, several additional observations about system scalability can be made:

1. **Scalability (1–8 Cores):** From 1 to 8 threads, speedup increases significantly, peaking at about $4.6\times$ for the S1S2 strategy. Although notable, this is still below the ideal $8\times$ linear scaling. The gap between $4.6\times$ and $8\times$ represents the cost of the non-parallelizable code fraction and the inherent OpenMP management overhead.

Threads	Speedup S1	Speedup S2	Speedup S1S2	Speedup S1S2 Opt
1	1.03×	1.04×	1.03×	0.97×
2	1.90×	1.01×	1.91×	1.87×
4	3.41×	1.02×	3.42×	3.37×
8	4.50×	0.99×	4.63×	4.59×
16	4.55×	1.02×	4.55×	4.50×

Table 1. Speedup results for the different configurations.

Threads	Efficiency S1	Efficiency S2	Efficiency S1S2	Efficiency S1S2 Opt
1	1.03%	1.04%	1.03%	0.97%
2	0.95%	0.51%	0.95%	0.94%
4	0.85%	0.26%	0.86%	0.84%
8	0.56%	0.12%	0.58%	0.57%
16	0.28%	0.06%	0.28%	0.28%

Table 2. Efficiency $E(N)$ for the different configurations.

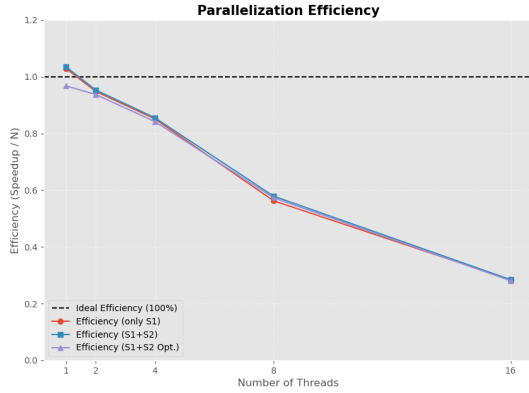


Figure 3. Efficiency as the number of threads increases.

2. Saturation and Degradation (8 vs. 16 Threads): The transition from 8 to 16 threads yields the most interesting insight. The test system has 8 physical and 16 logical cores (via Hyper-Threading). The data show that increasing from 8 to 16 threads actually *reduces* performance (speedup drops from $4.63\times$ to $4.55\times$). Figure 3 (Efficiency) makes this decline even clearer, with efficiency dropping from 58% to 28%. This experimentally confirms that, for this compute-bound workload, Hyper-Threading is counterproductive: the 16 logical threads cause excessive contention for physical resources (cache and execution units), and the overhead outweighs any potential benefit.

A particularly interesting observation comes

from comparing the **S1+S2** strategy with its optimized version. The theoretical expectation was that the optimized version—using a single parallel region and thus paying the *fork/join* overhead only once—would be faster. However, the experimental data show the opposite: the version using two separate *parallel for* directives is actually faster.

This suggests that the first version allows the OpenMP scheduler to better exploit data locality. At the start of the second loop, the scheduler appears “smart” enough to reassign the same boids to the same cores, which already have the relevant data in cache (a cache hit). In contrast, the “optimized” version, in attempting to balance the workload within a single parallel region, likely disrupted this locality, causing frequent cache misses. This experiment demonstrates that, for this problem, the benefit of cache reuse (approximately 1.7 s) outweighed the cost of paying an additional thread wake-up overhead.

Finally, a comparison was made between **static** and **dynamic** scheduling policies to check for further improvements. The hypothesis was that the $O(N^2)$ workload is homogeneous and perfectly balanced, as each boid performs an identical amount of computation. Therefore, the *schedule(static)* policy—dividing the work into N equal chunks—should be the most efficient. Experimental results confirmed this hypothesis: under the best-performing configuration (8

Threads	Time Seq (s)	Time S1 (s)	Time S2 (s)	Time S1S2 (s)	Time S1S2 Opt (s)
1	766.622	745.178	740.144	740.701	792.028
2	740.500	404.176	758.695	402.032	408.934
4	740.543	225.111	749.691	224.072	227.636
8	742.956	170.289	773.913	165.459	167.154
16	742.750	168.415	751.627	168.390	170.546

Table 3. Execution times (in seconds) for the different configurations.

threads), dynamic scheduling produced a run-time of 170.8 s compared to 165.4 s with static scheduling, making it slightly slower. This is because dynamic scheduling is designed for unbalanced workloads and introduces

6. Conclusions

The Boids simulation was successfully parallelized. The **S1+S2** strategy (two separate *parallel for* regions) achieved the best performance, reaching a speedup of approximately $4.6\times$ on 8 cores, in accordance with Amdahl's Law. Furthermore, the results demonstrated that CPU cache management has a greater impact on performance optimization than the cost associated with thread creation and synchronization overhead.

References

- [1] Craig W. Reynolds. Flocks, herds, and schools: A distributed behavioral model. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '87, pages 25–34, New York, NY, USA, 1987. ACM.
- [2] V. Hunter Adams. Boids algorithm. Personal Web Site.